МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ЛАБОРАТОРНОЙ РАБОТЕ

И1

Параллельный генетический алгоритм

IV. Интеллектуальные вычисления • Базовый уровень сложности

Среда: Windows 10 · MSVC C++17 · OpenMP · NVIDIA CUDA · Python / Matplotlib

УСЛОВИЯ ЛР № 0 выполнена, средства CPU/OpenMP/CUDA установлены и проверены. При отсутствии совместимой NVIDIA на личном ноутбуке GPU-эксперименты проводятся на компьютере учебного класса.

Суть исследования

Разработать простой генетический алгоритм для поиска минимума многомерной функции; реализовать последовательную оценку популяции на CPU, её многопоточную версию с OpenMP и массово-параллельную оценку на GPU с CUDA. Сопоставить время вычислений, полное время алгоритма и качество найденного решения.

Прикладное значение

Популяционные методы применяют для поиска параметров моделей, настройки инженерных и финансовых систем, оптимизации расписаний, размещения ресурсов и других задач, где множество допустимых решений велико. Параллельные вычисления особенно полезны, когда оценка одного кандидата дорогостоящая (например, требует моделирования или обучения модели).

Что надо сдать

- Исходный код: последовательная CPU-реализация, CPU/OpenMP и CUDA-версия вычисления функции качества; README с командами сборки и запуска.
- Автоматические тесты функции и корректности работы алгоритма; документированная схема генерации случайных чисел.
- CSV с исходными результатами экспериментов (не только усреднениями), графики скорости и качества, аналитический отчёт.
- Устная защита: объяснение операций генетического алгоритма, распараллеливания, границ честного сравнения CPU и GPU.

1. Цель, результаты обучения и границы работы

1.1. После выполнения лабораторной студент умеет (мы надеемся)

- Объяснять понятия популяции, особи, генома, целевой функции, поколения, селекции, скрещивания, мутации и элитизма.
- Реализовывать генетический алгоритм с числовым геномом и контролируемой генерацией случайных чисел.
- Выделять независимую оценку кандидатов как участок, допускающий параллельное выполнение без блокировок.
- Писать параллельный цикл OpenMP и CUDA-ядро «один поток — одна особь».
- Разделять скорость одной оценки, полное время поколений и качество оптимизации.
- Выполнять серии измерений, рассчитывать ускорение, строить графики и обосновывать результаты.

1.2. Что не требуется

Не требуется реализовывать промышленный генетический оптимизатор, сложную островную модель, параллельную селекцию на GPU, несколько способов кодирования генома или обучение нейросетей. Всё это оставлено для работ И2–И3.

2. Математическая постановка

2.1. Целевая функция Растригина

Найдите минимум функции D переменных на гиперкубе $x_i \in [-5{,}12; 5{,}12]$:

$$f(x) = 10D + \Sigma_{i=1}^{D} [x_i^2 - 10\cos(2\pi x_i)].$$

Известный глобальный минимум: f(0, …, 0) = 0. Многочисленные локальные минимумы затрудняют оптимизацию; функция удобна для сопоставления разных стратегий поиска. В работе применяется минимизация f (меньшее значение — лучше).

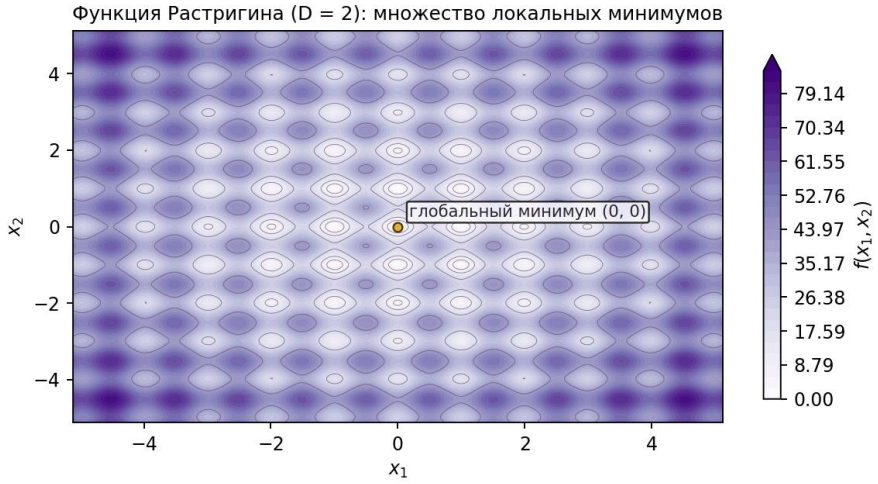


Рис. 1. Ландшафт функции для двух переменных; при D > 2 форма пространства усложняется.

2.2. Структура генетического алгоритма

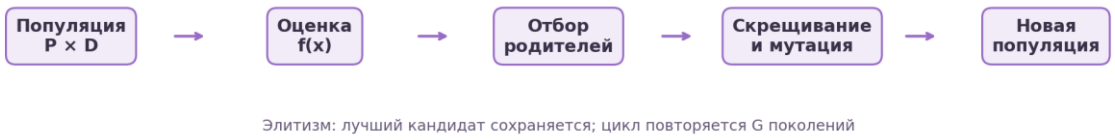


Рис. 2. Схема одного поколения генетического алгоритма.

Одна особь — вектор D вещественных чисел. Популяция содержит P особей; число поколений G. На каждом поколении для всех особей вычисляется значение f, затем формируется новая популяция.

2.3. Единый вариант операторов (обязательный)

Оператор	Принятая схема
Инициализация	P случайных векторов; каждая координата равномерно выбирается из [−5,12; 5,12].
Отбор	Турнир из k = 3 случайных участников (минимальное f выигрывает); выбирать с возвращением.
Скрещивание	Арифметическое: ребёнок c = λa + (1−λ)b, где λ ∈ [0; 1] выбирается случайно.

Оператор	Принятая схема
Мутация	Для каждого гена вероятность $p_m = 1/D$; при мутации прибавить шум $N(0; \sigma^2)$, где $\sigma = 0{,}20$.
Границы	После мутации обрезать координаты до диапазона $[-5{,}12; 5{,}12]$.
Элитизм	Лучшая особь поколения копируется в следующее поколение без изменения.
Критерий завершения	Ровно G поколений; досрочную остановку в базовой работе не применять.

ЛОВУШКА Процедура отбора должна корректно сравнивать минимальные значения f. Переменные fitness, score и objective легко перепутать: здесь оптимизируемая величина f уменьшается, а не увеличивается.

2.4. Простейшие проверки функции

Вход (D = 2)	Ожидаемое значение f	Почему
(0; 0)	0	Глобальный минимум
(1; 0)	1	$\cos(2\pi) = 1$
(1; 1)	2	Сумма квадратов целых координат
(0,5; 0)	20,25	$\cos(\pi) = -1$

Точность сравнения результатов CPU и GPU: использовать double во всех трёх версиях. Для промежуточных значений f допускается абсолютная погрешность 10^{-8} или относительная 10^{-10} (в зависимости от масштаба значения); требовать побитового совпадения результатов не нужно.

3. Организация исходных данных и воспроизводимость

3.1. Представление популяции

Для CPU и GPU использовать одинаковый линейный массив data размером P × D: координата j особи i хранится по индексу data[i·D + j]. Значения f хранить отдельно в массиве scores длины P. Такой формат удобен для копирования на GPU и для параллельной обработки независимых особей.

```
// Схема хранения (фрагмент, не полный алгоритм)
std::vector<double> population(P * D);
std::vector<double> scores(P);
// Координата j особи i: population[i * D + j]
// Значение качества i:      scores[i]
```

3.2. Генераторы случайных чисел

Инициализацию популяции, турнирный отбор, коэффициенты скрещивания и мутации выполнять на CPU, используя std::mt19937_64 с явным seed. Не допускать одновременного использования одного общего генератора из нескольких потоков. В обязательной версии OpenMP и CUDA распараллеливается только вычисление f; остальные операторы остаются одинаковыми по смыслу на CPU.

Один и тот же seed следует использовать для сопоставления реализаций, а для исследования устойчивости — серию разных seed. Незначительные отличия арифметики CPU/GPU могут изменить ход эволюции после выбора родителей; поэтому строгую проверку численной корректности проводят на одном заранее заданном массиве кандидатов, а качество генетического поиска анализируют статистически.

4. Обязательная программная часть

4.1. Этап A — последовательный CPU

1. Реализовать rastrigin(data, i, D): вычисление f для одной особи.
2. Реализовать оценку всех P особей в последовательном цикле.
3. Реализовать инициализацию, турнирный отбор, скрещивание, мутацию, элитизм и обновление популяции.
4. По окончании каждого поколения сохранять best_f; проверять конечность значений и принадлежность координат допустимому диапазону.

```
for (int i = 0; i < P; ++i) {
    scores[i] = rastrigin(population.data() + i * D, D);
}
// Далее: элитизм, отбор, скрещивание, мутация.
// Эти операции студент реализует самостоятельно.
```

4.2. Этап B — OpenMP на CPU

Распараллелить только независимый цикл оценки популяции. У каждого i — собственный участок чтения population и собственная ячейка scores[i]; поэтому для записи результатов не нужны mutex, critical или atomic. Количество потоков передавать параметром командной строки.

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < P; ++i) {
    scores[i] = rastrigin(population.data() + i * D, D);
}
```

Выполнить тесты с 1, 2, 4, 6, 8 и 12 потоками (если процессор поддерживает соответствующее количество логических потоков). По желанию изучить schedule(dynamic), объяснив, есть ли основания ожидать его преимущества при одинаковой D и одинаковой трудоёмкости каждой особи.

4.3. Этап C — CUDA GPU

Реализовать ядро, в котором один GPU-поток вычисляет f ровно для одной особи. Если глобальный индекс i ≥ P — выйти. Для базовой реализации использовать double; каждая особь занимает непрерывный отрезок из D чисел.

```
__global__ void evalRastrigin(
    const double* data, double* scores, int P, int D) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i >= P) return;
    // TODO: вычислить f(data[i*D ... i*D+D-1])
    // TODO: записать результат только в scores[i]
}
```

В основной программе: выделить память cudaMalloc для data и scores; перенести данные cudaMemcpy Host→Device; запустить CUDA-ядро; дождаться завершения и проверить ошибки; скопировать scores обратно cudaMemcpy Device→Host. Выделение буферов желательно выполнять один раз перед серией поколений, а освобождение — после её завершения.

Для производительности проверить blockDim = 64, 128, 256 и 512 (в пределах аппаратных ограничений); число блоков вычислять через округление вверх (P + blockDim − 1) / blockDim. Не считать запуск ядра синхронным: он асинхронен относительно CPU.

4.4. Что именно должно совпадать

- Результат rastrigin для набора фиксированных особей на CPU и GPU в установленной погрешности.
- Размеры и границы всех массивов, отсутствие CUDA-ошибок, NaN и Inf.
- Инвариант элитизма: best_f не должен возрастать от поколения к поколению (для каждого отдельного запуска).

- Для CPU-последовательной и CPU/OpenMP реализации при одинаковом seed траектория best_f должна совпадать, если генерация и выбор родителей выполняются в том же порядке.

5. Программа экспериментов: время, качество, масштабируемость

5.1. Матрица вычислительных размеров

Набор	P — особей	D — координат	G — поколений	Назначение
S	512	8	30	Отладка; накладные расходы
M	4096	32	30	Основное сравнение
L	16384	64	30	Проверка масштабируемости

Дополнительно (по желанию): изменить P при фиксированном D, а затем D при фиксированном P, чтобы раздельно оценить влияние размера популяции и размерности задачи. Нельзя сравнивать разное число поколений G как одинаковый объём вычислений.

5.2. Разделение измерений

Метрика	Определение
T_eval	Время вычисления f для одной фиксированной популяции без генерации новой.
T_kernel	CUDA Events: только выполнение GPU-ядра для оценки f.
T_gpu_eval_total	Копирование data Host→Device + ядро + копирование scores Device→Host; выделение памяти не включать, если оно не включено в аналогичный CPU-тест.
T_ga_total	Полное время G поколений: операции генетического алгоритма + все оценки и передачи данных; без вывода файлов на диск.
S_p	Ускорение: T_CPU_seq / T_variant при одинаковом содержании измеряемого участка.
Q	Качество: лучшее найденное значение f после G поколений.

ЧЕСТНОЕ СРАВНЕНИЕ Нельзя делить время всей CPU-программы на одно лишь время CUDA-ядра и называть результат «ускорением программы»: это разные границы измерения. Сравнивать T_eval и T_kernel можно только как характеристику вычислительной фазы с явной оговоркой.

5.3. Повторения и фиксация seed

1. Для микробенчмарка оценки одной заранее созданной популяции: прогрев; минимум 10 повторов, одинаковый массив; медиана и разброс времени.
2. Для полной программы: одинаковые P, D, G и seed для CPU/GPU, не менее 5 измерений каждого варианта. Порядок серий перемешивать.
3. Для статистики качества решения: не менее 10 независимых seed (например, от 100 до 109); приводить медиану, лучший результат и межквартильный размах Q.
4. Во всех измерениях сохранять исходные отдельные запуски; исключить консольный вывод из измеряемой области.

5.4. Шаблон записи измерений CSV

```
timestamp,device,variant,P,D,G,threads,block_size,seed,repeat,
metric,time_ms,best_f
2026-10-18,CPU,seq,4096,32,30,1,0,100,1,ga_total,0.0,0.0
2026-10-18,CPU,openmp,4096,32,30,4,0,100,1,ga_total,0.0,0.0
2026-10-18,GPU,cuda,4096,32,30,0,256,100,1,ga_total,0.0,0.0
```

Строки выше — только пример структуры; нулевые значения должны быть заменены реальными результатами. Для измерения T_kernel используйте CUDA Events с синхронизацией события окончания; для T_ga_total — std::chrono::steady_clock на стороне CPU. Указывайте версии компилятора, драйвера и параметры оптимизации в паспорте эксперимента.

5.5. Обязательные графики

1. T_eval от P при фиксированном D: последовательный CPU, OpenMP и CUDA (отдельно ядро и полный перенос).
2. T_ga_total от P при одинаковых D, G и seed: три реализации.
3. Ускорение OpenMP в зависимости от числа потоков для одного набора данных.
4. Динамика best_f по поколениям (не менее трёх seed), отдельно для одного выбранного режима.
5. Зависимость конечного best_f от затраченного времени: точки для независимых запусков и/или сводная диаграмма распределения по seed.

Дополнительно рекомендуется показать влияние blockDim на T_kernel и сравнить эффективность версии с double и экспериментальной версией с float, если удастся сохранить осмысленный уровень точности.

6. Проверка корректности и типовые ошибки

Проверка	Ожидаемый результат
P = 1, D = 2	Оценка одной особи корректна; отбор не обращается за пределы массива.
D = 1	Вероятность мутации $p_m = 1$; алгоритм не делит на ноль.
P не кратно размеру CUDA-блока	Лишние GPU-потоки безопасно завершаются по i ≥ P.
Крайние значения −5,12 и 5,12	Целевая функция конечна; операция clamp сохраняет границы.
Повтор одного seed	CPU-последовательная версия воспроизводит ту же последовательность результатов.
Best-of-generation	При элитизме лучший достигнутый f не ухудшается.

6.1. Частые ошибки

- Неверная направленность селекции: выбирается максимальное f вместо минимального.
- Использование одного генератора случайных чисел несколькими потоками без защиты.
- Перемещение CUDA-буферов и выделение памяти внутри измеряемого ядра.
- Гонка за общий best_f при параллельной оценке популяции.
- Измерение GPU без cudaDeviceSynchronize или корректных CUDA Events.
- Сравнение качества по одному случайному запуску; оценка GPU без времени копирования.
- Неэквивалентность настроек операторов и числовых типов между версиями.

7. План выполнения

1. Сдать автоматические тесты функции Растригина и проверки базовых граничных условий.
2. Подготовить последовательный генетический алгоритм и показать улучшение лучшего результата на нескольких seed.
3. Распараллелить оценку особей через OpenMP; проверить равенство результатов на фиксированной популяции.

4. Создать CUDA-ядро и проверить результаты против CPU-эталона.
5. Собрать таблицу измерений; построить пять обязательных графиков.
6. Сформулировать минимум три наблюдения, подкреплённых числами, и минимум два ограничения исследования.
7. Оформить README, приложить исходный CSV и защитить работу.

8. Отчёт и оценивание

8.1. Структура отчёта

- Титульные сведения, цель, конфигурация CPU/GPU и используемое программное обеспечение.
- Математическая постановка, значения параметров GA и блок-схема основных этапов.
- Описание трёх реализаций и распределения вычислений между CPU и GPU.
- Проверка корректности с числовыми примерами; методика измерений.
- Таблицы измерений, графики, сравнение скорости и качества решений.
- Выводы: когда распараллеливание выгодно, где находится узкое место, как ограничения эксперимента влияют на результат.

8.2. Критерии защиты

Критерий
Корректность GA, операторов, граничных условий и тестов
Последовательная CPU + OpenMP + CUDA реализации
Экспериментальная дисциплина, CSV, корректное измерение времени
Графики, интерпретация ускорения и качества, воспроизводимость
Устная защита и понимание ограничений

КРИТЕРИЙ ЗАЧЁТА Лабораторная И1 относится к базовому уровню индивидуальной траектории. Результат учитывается в общей системе 4 × 3 работ; даже базовая работа требует работоспособных CPU/OpenMP/CUDA вариантов, проверки корректности и интерпретации измерений.

9. Вопросы для поднятия самооценки

1. Чем целевая функция отличается от функции приспособленности? Почему здесь выбирают минимум?
2. Что такое геном, популяция, поколение и элитизм?
3. Почему значение f(0, …, 0) равно нулю и почему это полезно для тестирования?
4. Почему функция Растригина считается мультимодальной?
5. Как работает турнирный отбор и чем управляет его размер k?
6. Какие преимущества и ограничения имеет арифметическое скрещивание вещественных векторов?
7. Как влияет вероятность мутации $p_m = 1/D$ при изменении размерности D?
8. Почему после мутации требуется обработка границ допустимой области?
9. Какая часть алгоритма распараллелена в базовой версии и почему именно она?
10. Почему OpenMP-цикл оценки не требует блокировки для массива scores?
11. Почему общее состояние RNG нельзя бездумно использовать из нескольких потоков?
12. Что означает «один CUDA-поток — одна особь» и как обрабатывается неполный блок?
13. Почему T_kernel, T_gpu_eval_total и T_ga_total могут сильно различаться?
14. Почему большее количество CPU-потоков не гарантирует ускорение?

15. Почему CUDA может проиграть CPU при небольшой популяции?
16. Почему сравнение окончательного best_f по одному seed недостаточно?
17. Что означает «честное сравнение» двух реализаций при разных числовых округлениях?
18. Можно ли одновременно улучшить качество оптимизации и ухудшить время её выполнения?
19. Как отличается ускорение вычисления f от ускорения всей генетической программы?
20. Какое преимущество дал бы GPU, если оценка одной особи предполагала запуск сложной модели?

10. Дополнительные задания (необязательно)

- Построить график Парето «время / найденное значение f» для разных размеров популяции.
- Перенести операторы селекции и скрещивания на GPU; обсудить компромисс между копированием данных и сложностью реализации.
- Сравнить static и dynamic OpenMP scheduling при неодинаковой трудоёмкости функций качества (в отдельной дополнительной модели).
- Реализовать альтернативную целевую функцию Розенброка и сравнить особенности сходимости.
- Сравнить double и float на GPU: производительность, погрешность f и влияние на траекторию поиска.

11. Полезные официальные источники

1. NVIDIA. CUDA Programming Guide — kernel launch, streams, events. https://docs.nvidia.com/cuda/cuda-programming-guide/

2. OpenMP Architecture Review Board. OpenMP Specifications / Examples. https://www.openmp.org/specifications/

3. Microsoft. C/C++ Building Reference / OpenMP with MSVC. https://learn.microsoft.com/en-us/cpp/parallel/openmp/openmp-in-visual-cpp

4. NVIDIA. CUDA Runtime API (CUDA Events и синхронизация). https://docs.nvidia.com/cuda/cuda-runtime-api/

Материал предназначен для учебного практикума. Все числовые результаты измерений студент получает самостоятельно; примеры в методичке не являются готовыми результатами экспериментов.